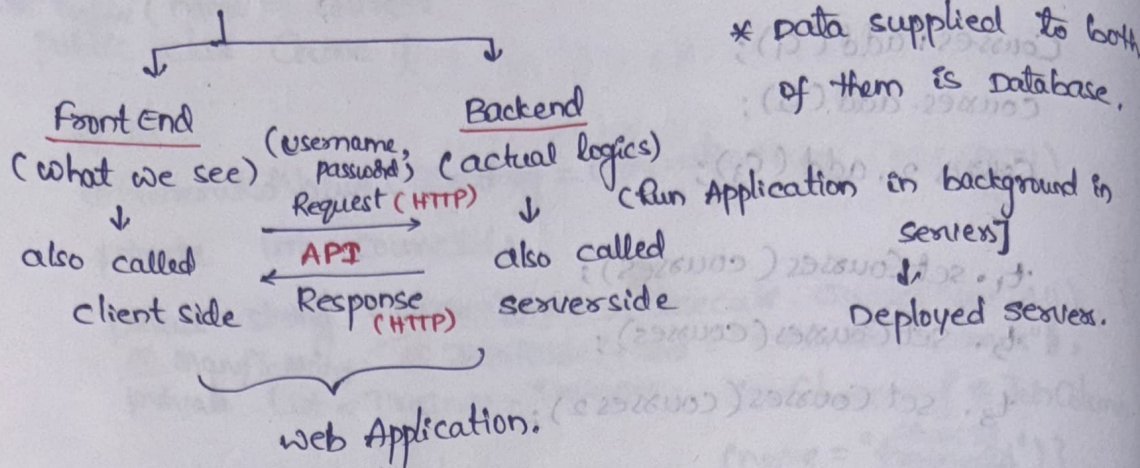


SERVLETS & DYNAMIC WEB APPLICATIONS

Java FullStack



- * **API** - Application Programming Interface.
- * **HTTP** - Hypertext Transfer Protocol - This will safely carry request from client to server. (It is not completely secure)
- * **HTTPS** - Hypertext Transfer Protocol Secure
- * **DNS**
- * **SERVER**

Ex: **Tomcat**, **GlassFish**, **Jetty**, **Netty**, **Jboss**.

* Dynamic Web Applications:

→ Data changes regularly.

* SERVLETS:

- All APIs handled by servlets.
- All Requests & Responses b/w Frontend & Backend handled by servlets.

API → Request → Servlets → Java
 Ex: (Hotel waiter) → (corder) → (mediator) → (chef)

* **Server** is a servlet container.

* Client → Request → Server → Pick a servlet → Handover to actual Java code.

* Servlets: 2

1. Generic servlets
2. Http servlets

J2EE
 ↓
 * **JavaEE** - Java Enterprise Edition.
 ↓
 JavaX - Tomcat 9
 ↓
 Jakarta - Tomcat 10.

* Tomcat 10 → supports Jakarta.

Ex: // to build an Application.

1. Open Eclipse → New → other → Dynamic Web Project → Next →
project Name → Target runtime (None Apache Tomcat 4.1) → Dynamic
web module Version (2.0) → Config Finish → src/main/java → Next →
→ Configure web module settings → ☒ → Finish

> src

> main

□ Java
 > com

✱ webapp

> META-INF

> WEB-INF ✓

> lib

> web.xml

Ex: New → other → servlet → Next → Java Package → className
(class)

Java

> com → class other → class → package → Name.

package com;

public class FirstServlet extends HttpServlet {

// Window → show view → servers → click on link → tomcat (10.1) server

→ Next → ~~Finish~~ → Tomcat server → Browse → C: Drive → Program

files → Apache software Foundation → Tomcat 10 → Finish → Finish.

// Rightclick on project → Targetted Runtimes → Apache Tomcat (10.1v) →

Apply & close (Now we can get HttpServlet (jakarta.servlet...))

@Override

```
protected void doGet (HttpServletRequest req, HttpServletResponse  
resp) throws ServletException {  
    response.setContentType("text/html");  
    PrintWriter writer = resp.getWriter(); (write something  
    writer.append("Hi... Welcome Dude!!!");      - ng into brow  
    ser)  
}
```

}

// to register servlets.

web.xml $\Rightarrow$ deployment descriptor (to deploy servlet into server)

tags: $\langle$ web-app $\rangle$

$\langle$ servlet $\rangle$

$\langle$ servlet-name $\rangle$ hello $\langle$ /servlet-name $\rangle$

$\langle$ servlet-class $\rangle$ ^{com.} FirstServlet $\langle$ /servlet-class $\rangle$

(packagename).(className)

$\langle$ /servlet $\rangle$

$\langle$ /servlet-mapping $\rangle$

$\langle$ servlet-name $\rangle$ hello $\langle$ /servlet-name $\rangle$

$\langle$ url-pattern $\rangle$ /hello $\langle$ /url-pattern $\rangle$
(s) hi

$\langle$ /servlet-mapping $\rangle$

$\langle$ /web-app $\rangle$

// to run the project,

$\rightarrow$ Rightclick on project $\rightarrow$ Run As $\rightarrow$ Run on Server $\rightarrow$ Tomcat v10.1

Next $\rightarrow$ (Configured) $\rightarrow$ Finish.

* Doubleclick on servers $\rightarrow$ Tomcat adminport $\rightarrow$ 8081 $\rightarrow$ Enter

** (chrome)

* Browser $\rightarrow$ http://localhost:8080/FirstDynamicWebProject/hi

* Every process run on a port.

* Similarly, Tomcat also run on port - 8080

* Servlets:

Types - 2

1. Generic Servlet

2. HTTP Servlet.

1. Generic Servlet:

* It is a common servlet.

* It supports any type of protocols.

* It is not safe.

2. Http Servlets:

→ This works completely only on Http protocol.

→ This is safe.

* Servlet Life cycle: 3 steps1. init()2. Service()3. Destroy()1. init() - Initialization→ init() → this method automatically called (It works only one time).

→ When we

2. Service() - Actual execution of servlet.Service Methods

HTTP Methods → doGet() → service method. - When we want to see data.

↓

Used → doPost() → Saving → when user send some data (save new data).

↓

→ doPut() → Update the existing data.

↓

→ doDelete() → delete the data.

in APIs.

3. Destroy() -

- When the servlet is closed, it destroyed in memory.
- Shutdown your server
- Refresh - Redeployed (existing servlets are destroyed & new servlets are formed).

Ex:

Create Dynamic webproject → Projectname → Next → Create ^{web.xml} sample

Server → web.xml →

Frontend:

Html, CSS, Javascript, ReactJS

* **Html**: Hyper Text Markup language.

- used to show ~~browser~~ something on to browser.
- It will give skeleton

Java Script:

Actual functionality.

* All markup languages have tags.

Html: `<html>` - Starts with it - It is a text format.

* Heading written in `<h>` `</h>` tag.

* Normal text written in `<p>` `</p>` tag.

* Images added in `<img>` `</img>`

* Add some links `<a>` `</a>` (anchor tags)

→ `<head>` - head tag

→ `<body>` - body tag. [actual content written in it]

→ `<table>` - table tag

→ `<form>` tag - signup, login etc.

web.xml:

* `<welcome-file-list>`

`<welcome-file>` index.html `</welcome-file>`

`</welcome-file>`

* Client → submit → server → servlet → doGet/doPost →

request, response → data → request.

* create servlet

package com;
@WebServlet("/HiServlet") → [we no need to use web.xml if we use this annotation]
public class HiServlet extends HttpServlet {

```
protected void doGet (HttpServlet Request request,  
                        HttpServletResponse response) {  
    PrintWriter writer = response.getWriter();  
    response.setContentType ("text/html");  
    writer.append ("Hi"); → Hi  
    writer.append (" <h1> Hi Dude ... </h1>"); → Hi Dude...  
    writer.append (" <h1> Hi Dude ... </h1> <br/> " +  
                  " <h3> This is H3 </h3>"); → Hi Dude...  
                                                this is H3
```

Html file creation:

webapp (rightclick) → New → other → html file / file. index.html (index.html) → finish.

```
<head>  
* <title> My Page </title> (Changes file name in browser)  
<body>  
  <h1> My First HTML Page </h1>  
  <a href = "hello" > sign up </a>  
  <body>  
  </html>
```

*
@WebServlet ("/hello")
public class HelloServlet extends HttpServlet {
 protected void doGet (-----) { "signup.html"
 RequestDispatcher = request.getRequestDispatcher ("hello.html") → Html file name.
 requestDispatcher.forward (request, response);
 }
}

html file: `<html>`

hello.html

`<head>`

`<title> Hello </title>`

`</head>`

`<body>`

`<h1> welcome to FLM </h1>`

`<p> JFS - Java Fullstack </p>`

`<marquee> Enroll Now Last Date Approaching </marquee>`

`</body>` (scrolls some data)

`</html>`

⇒ This file loads when we forward servlet into html file

webapp - Signuppapp - Signup.html

`<html>`

`<head>`

`<meta>`

`<title> Signup </title>`

`</head>`

`<body>`

`<h1> Instagram </h1>`

`<form action = "signup" method = "post"`

(accepts data) (servlet url)

`<input type = "text" name = "username">`

`<input type = "text" name = "password">`

`<input type = "submit" value = "signup">`

`</form>`

Signup servlet: `</body>`
`</html>`

@WebServlet("/signup")

public class SignupServlet extends HttpServlet {

protected void doPost (... request, ... response) {
(sending some data)

* server start → index.html → anchor → hello(servlet) → signup

String username = request.getParameter("username");

String password = request.getParameter("password");


```
printWriter writer = response.getWriter();
```

```
response.setContentType("text/html");
```

```
writer.append("Hi" + username + " You are successfully  
registered");
```

```
}
```

29/07/2025

class-66

* HTML pages are called static page (we can't change data dynamically).

* HTML cannot send dynamic data. (we can't load dynamic data).

* JSP: Java Server Pages.

(outdated) (we write code in server side using Java)

→ It is a combination of Java & HTML.
(Backend) (Frontend).

→ Now, Java & HTML running in different servers & b/w them there are APIs. deployed

→ In JSP → Java & HTML deployed in same server.

1. Scriptlet:

* To write Java in ~~HTML~~ ^{JSP} - Scriptlet → `<% %>`

* To print java code → `out.println()`
Ex: `<% int a = 10;
out.print(a);
%>`

2. Expression:

→ This is explicitly used to print something by using tag,

Tag:- `<% = a %>` → It prints 'a'
↓
(printed variable)

3. Directive:

→ It is a kind of metadata (like web.xml file)

→ It is a indication that tells the browser (8) JSP Engine that how to process the page.

Tag: `<% @ %>`

* Imports in Java, will have to be imported in directive

Ex: Project creation:

⇒ Dynamic Web Project

⇒ Webapp ⇒ index.html (HTML file)

```
<html>
```

```
<head>
```

```
<meta
```

```
<title> Home </title>
```

```
</head>
```

```
<h1> welcome to JFS... </h1>
```

```
</body>
```

```
</html>
```

Servlet creation → src > java

```
package com; Servlet URL
```

```
@WebServlet("/hello")
```

```
public class HelloServlet extends HttpServlet {
```

```
protected void doGet (--- request, --- response) {
```

```
    RequestDispatcher requestDispatcher = request.getRequestDispatcher("hello.jsp");
```

RequestDispatcher

```
    requestDispatcher.forward(request, response);
```


JSP file creation:

webapp → JSP file → hello.jsp

⇒ JSP page loads

```
<%@ page language = "java" contentType = "text/html"
```

```
<body>
```

```
<h1> This is JSP Page </h1>
```

```
<% String name = "Ravi";
```

→ **scriptlet**

⇒ It works as Java.

```
out.println(name);
```

```
%>
```

```
<%= name %>
```

→ **expression.**

```
</body>
```

```
<h1> Hi <%= name %> You are a good Tutor </h1>
```

⇒ Hi Ravi You are a good Tutor.

* To get name & password using JSP

⇒ Singup Servlet
Request Dispatcher = request. ^{Request}getDispatcher("result.jsp");
requestDispatcher request.set ^{key}Attribute("name", ^{value}username);
request.set ^{key}Attribute("password", ^{value}pass);
requestDispatcher.forward(request, response);

JSP file:

[* If we want to send some data into JSP we use setAttribute() method.]

(we can get data dynamically.)

result.jsp

<body>
<%

// String name = request.getAttribute("name");
(It always returns object type)

String name = (String) request.getAttribute("name");

String password = (String) request.getAttribute("password");

<h1> Hi <%= name %> You have Signed Up-- </h1>

<h2> Don't expose your Password : h2 <%= password %>
</h2>

* [

x scopes:

1. request → It is available once per Http Request.
(like private).
 2. session → Data is available across the session.
(like default)
Ex: Chrome → /signup → servlet → session → access in any JSP.
(session provides)
- Once we move to other session there is no available of data.
3. Application/ server → Available across the server in all sessions.
(like public)

Ex:

```
@webServlet("/signup")
public class TestServlet extends HttpServlet {
    protected void doPost(--- request, --- response) {
        String name = request.getParameter("username");
        String password = request.getParameter("password");
        request.setAttribute("name", name);
        // request.setAttribute("password", password);
        // request.getSession()
        RequestDispatcher requestDispatcher = request.getRequestDispatcher("result.jsp");
        requestDispatcher.forward(request, response);
        // session scope.
        HttpSession session = request.getSession();
        session.setAttribute("password", password);
        // Application scope.
        ServletContext servletContext = request.getServletContext("name", name);
    }
}
```


index.html

<h1> welcome to JFS </h1>

<form action = "signup" method = "post">

<input type = "text" name = "username" >

<input type = "password" name = "password" >

<input type = "submit" value = "signup" >

</form>

result.jsp

<title> Result </title>

<body>

<h1> name : <% = ^{application} request.getAttribute("name") %> </h1>

<h1> password : <% = ^{session} request.getAttribute("password") %> </h1>

result2.jsp

// session scope

<h1> name : <% request.getAttribute("name") %> </h1>

<h1> password : <% session.getAttribute("password") %> </h1>

// application Scope

<h1> name : <% application.getAttribute("name") %> </h1>

<h1> password : <% ^{application} session.getAttribute("password") %> </h1>

servlet

@WebServlet("/hi")

public class HiServlet extends HttpServlet {

protected void ~~HiServlet~~ ~~exte~~

doGet (--- request, --- response) {

RequestDispatcher requestDispatcher = request.getRequestDispatcher("result.jsp")

requestDispatcher.forward(request, response);

(Even if we not attribute data, we get username & password due to session scope).

Application scope - access in different browsers)

— x —

PROJECT (Real Time Web Application)

PROJECT: Dynamic Web Project -

DATABASE

→ Employee Management

→ Users Table

→ FLM Admin

→ Employee Table

→ Signup

→ Login

⇒ Employees

→ database.

⇒ Add Employee

→ Form (Employee details)

→ Add

STEPS

⇒ 1. Dynamic web Project

2. JSP

3. Servlet

4. Data base - connection

5. JDBC / Hibernate.

6. MySQL Connector (dependency)